

✓ Welcome To Train Your Model

```
print("Welcome")
```

Welcome

Upload Your File

```
from google.colab import files
uploaded = files.upload()
```

[Choose Files](#) table6_2026_04.csv

table6_2026_04.csv(text/csv) - 580477 bytes, last modified: 8/28/2026 - 100% done
Saving table6_2026_04.csv to table6_2026_04.csv

Create Data frame

```
import pandas as pd
import numpy as np
df = pd.read_csv("table6_2026_04.csv")
print(df.shape)
# verify
df.head(1)
```

(1981, 18)

	reporting_month	ministry	sector	sl_no	project_name	agency	pro
0	2026-04	Ministry of Civil Aviation	Aviation & Aviation Infrastructure	1	Construction of New Domestic Terminal Building...	Airport Authority of India [AAI]	

Next steps:

[Generate code with df](#)[New interactive sheet](#)

Verify Data First

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1981 entries, 0 to 1980
```

Data columns (total 22 columns):

#	Column	Non-Null	Count	Dtype
0	reporting_month	1981	non-null	object
1	ministry	1981	non-null	object
2	sector	1981	non-null	object
3	sl_no	1981	non-null	int64
4	project_name	1981	non-null	object
5	agency	1981	non-null	object
6	project_code	1981	non-null	int64
7	legacy_ocms_code	1184	non-null	object
8	pmgid	1213	non-null	float64
9	state	1981	non-null	object
10	approval_start_date	1970	non-null	datetime64[ns]
11	revised_start_date	1981	non-null	datetime64[ns]
12	target_doc	1981	non-null	object
13	revised_doc	1627	non-null	object
14	original_cost_cr	1981	non-null	float64
15	revised_cost_cr	1981	non-null	float64
16	cumulative_expenditure_cr	1981	non-null	float64
17	physical_progress_pct	1981	non-null	float64
18	financial_progress_pct	1981	non-null	float64
19	progress_gap	1981	non-null	float64
20	start_delay_months	1970	non-null	float64
21	time_overrun	1981	non-null	int64

dtypes: datetime64[ns](2), float64(8), int64(3), object(9)
memory usage: 340.6+ KB

Check for Nan values

```
df.isna().sum()
```

	0
reporting_month	0
ministry	0
sector	0
sl_no	0
project_name	0
agency	0
project_code	0
legacy_ocms_code	797
pmgid	768
state	0
approval_start_date	11
revised_start_date	0
target_doc	0
revised_doc	354
original_cost_cr	0
revised_cost_cr	0
cumulative_expenditure_cr	0
physical_progress_pct	0
financial_progress_pct	0
progress_gap	0
start_delay_months	11
time_overrun	0

dtype: int64

✓ Feature Engineering

```
time_features = [
    "ministry",
    "sector",
    "agency",
    "state",
    "original_cost_cr",
```

```

        "cumulative_expenditure_cr",
        "physical_progress_pct",

        "financial_progress_pct",
        "progress_gap",
        "start_delay_months"
    ]

    target = "time_overrun"

```

Add Derived Features

```

# 1. Financial progress
df["financial_progress_pct"] = (
    df["cumulative_expenditure_cr"]
    / df["original_cost_cr"].replace(0, np.nan)
) * 100

```

```

# 2. Progress gap
df["progress_gap"] = (
    df["financial_progress_pct"]
    - df["physical_progress_pct"]
)

```

Clean the date time Formate

```

df["approval_start_date"] = pd.to_datetime(
    df["approval_start_date"], format="%m/%Y", errors="coerce"
)
df["revised_start_date"] = pd.to_datetime(
    df["revised_start_date"], format="%m/%Y", errors="coerce"
)

```

```

# 3. Start delay in months
df["start_delay_months"] = (
    (df["revised_start_date"].dt.year -
     df["approval_start_date"].dt.year) * 12
    +
    (df["revised_start_date"].dt.month -
     df["approval_start_date"].dt.month)
)

```

Check Null values of Features

```
print(df[time_features].isna().sum())
```

ministry	0
sector	0
agency	0
state	0
original_cost_cr	0

```

cumulative_expenditure_cr    0
physical_progress_pct        0
financial_progress_pct        0
progress_gap                 0
start_delay_months           11
dtype: int64

```

Convert dates into appropriate formate

```

df["approval_start_date"] = pd.to_datetime(
    df["approval_start_date"],
    format="%m/%Y",
    errors="coerce"
)

```

```

df["revised_start_date"] = pd.to_datetime(
    df["revised_start_date"],
    format="%m/%Y",
    errors="coerce"
)

```

Convert String to Numeric values

```

# Make physical progress numeric
df["physical_progress_pct"] = pd.to_numeric(
    df["physical_progress_pct"],
    errors="coerce"
)

```

✓ Prepare X and Y

Creat Time Over Run Columns

```

df["time_ouerrun"] = (
    df["revised_doc"] > df["target_doc"]
).astype(int)

print(df["time_ouerrun"].value_counts())

```

```

time_ouerrun
0      1252
1       729
Name: count, dtype: int64

```

prepare X and Y

```
X = df[time_features]
y = df["time_overrun"]

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
X shape: (1981, 10)
y shape: (1981,)
```

✓ Train and Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
X_train: (1584, 10)
X_test  : (397, 10)
y_train: (1584,)
y_test  : (397,)
```

✓ Preprocessing Pipeline

Define Time Model 's preprocessing columns

```
categorical_features = [
    "ministry",
    "sector",
    "agency",
    "state"
]

numeric_features = [
    "original_cost_cr",
    "cumulative_expenditure_cr",
    "physical_progress_pct",
    "financial_progress_pct",
    "progress_gap",
    "start_delay_months"
]
```

Requirements

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
```

```
numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
```

```
categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

Time Preprocessor

```
time_preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_features),
    ("categorical", categorical_pipeline, categorical_features)
])
```

✓ ML Model

Create Time Overrun Model

```
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline

time_model = Pipeline([
    ("preprocessing", time_preprocessor),
    ("model", LogisticRegression(
        class_weight="balanced",
        max_iter=1000
    ))
])
```

Train Time Overrun model

```
time_model.fit(X_train, y_train)

print("Time Overrun Model trained successfully.")
```

Time Overrun Model trained successfully.

✓ ML Model Prediction

```
y_pred = time_model.predict(X_test)
y_prob = time_model.predict_proba(X_test)[:, 1]

print(y_pred[:10])
print(y_prob[:10])
```

```
[0 0 1 0 1 0 0 0 1 1]
[0.13248139 0.37090538 0.72017166 0.19706719 0.73836435 0.11169658
 0.23768954 0.42418536 0.76104075 0.57958172]
```

✓ ML Model Evaluation

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

print("Classification Report:")
print(classification_report(y_test, y_pred))

print("ROC-AUC:")
print(roc_auc_score(y_test, y_prob))

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Classification Report:
              precision    recall  f1-score   support

         0       0.74      0.60      0.66         251
         1       0.48      0.64      0.55         146

   accuracy          0.61
  macro avg          0.61
weighted avg          0.65

ROC-AUC:
0.6492386617911914
Confusion Matrix:
[[150 101]
 [ 52  94]]
```

✓ Save and Download Model

Save

```
import joblib

joblib.dump(time_model, "time_overrun_model.pkl")
print("Time model saved successfully.")
```

Time model saved successfully.

Download
